RLS Policy Explanation

```
-- ====================================================
-- 8. Create Row Level Security Policies
-- ====================================================

-- HR Policy
CREATE POLICY hr_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'HR'
    AND department = current_setting('app.department', true)
);

-- Manager Policy
CREATE POLICY manager_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Manager'
    AND department = current_setting('app.department', true)
);

-- Employee Policy
CREATE POLICY employee_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Employee'
    AND emp_name = current_setting('app.username', true)
);

-- Admin Policy
CREATE POLICY admin_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Admin'
);
```

i did all this, so explain those codes we used for creating policies,
and how does the specific lines of codes shows only specific rows while hides others

Good 👌 this is the **core logic** of your Row Level Security (RLS) project. Let's break it down clearly and practically.

🔐 What Is Happening Here?

You created **Row Level Security policies** on the `employees` table.

That means:

> PostgreSQL automatically checks every row before returning it from `SELECT`.

If the `USING` condition is TRUE → row is visible.
If FALSE → row is hidden.

So PostgreSQL internally behaves like:

```
SELECT * FROM employees
WHERE <policy condition>;
```

You didn't write that WHERE clause manually — PostgreSQL adds it automatically.

⚙️ Important Concept Used

🔹 `current_setting('app.role', true)`

This reads session variables you manually set:

```
SET app.username = 'hr1';
SET app.role = 'HR';
SET app.department = 'HR';
```

So each user session carries identity + role information.

Your policies compare table columns with these session values.

👩‍💼 1️⃣ HR Policy

```
CREATE POLICY hr_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'HR'
    AND department = current_setting('app.department', true)
);
```

🔎 What this means:

Row will be visible IF:

1. User role is `'HR'`
2. Employee's department = logged-in user's department

So internally:

```
SELECT * FROM employees
WHERE role = 'HR'
AND department = 'HR';
```

👉 Result:

HR can only see employees from their own department.

All other department rows → filtered out automatically.

👩‍💼 2️⃣ Manager Policy

```
CREATE POLICY manager_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Manager'
    AND department = current_setting('app.department', true)
);
```

🔎 Meaning:

Manager can only see employees in their department.

Same filtering idea — different role value.

If role ≠'Manager' → policy condition becomes FALSE
→ no rows pass this policy.

👷‍♀️ 3️⃣ Employee Policy

```
CREATE POLICY employee_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Employee'
    AND emp_name = current_setting('app.username', true)
);
```

🔎 Meaning:

Employee can only see:

```
WHERE emp_name = logged_in_username
```

So they can only view **their own row**.

All other rows fail the condition → hidden.

👑 4️⃣ Admin Policy

```
CREATE POLICY admin_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Admin'
);
```

🔎 Meaning:

Condition only checks:

```
role = 'Admin'
```

It does NOT restrict department.

So if role = Admin → condition TRUE for ALL rows.

Hence Admin sees everything.

🧠 Important: How PostgreSQL Applies Policies

If multiple SELECT policies exist:

PostgreSQL ORs them together.

Internally it becomes:

```
WHERE
   (HR condition)
OR (Manager condition)
OR (Employee condition)
OR (Admin condition)
```

But only one role matches at a time because of:

```
current_setting('app.role') = 'SomeRole'
```

So effectively only that policy activates.

🔥 Why Other Rows Are Hidden

For each row PostgreSQL checks:

Example: Logged in as Employee 'rahul'

For row with emp_name = 'sneha'

Policy check becomes:

```
'Employee' = 'Employee'   ✔
AND
'sneha' = 'rahul'         ❌
```

Since second condition FALSE → whole USING FALSE
→ row filtered out.

PostgreSQL simply does not return that row.

No error.
No warning.
It behaves like the row doesn't exist.

🎯 Why You Earlier Saw Rows Without Policy

RLS only works if:

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
```

If not enabled → policies exist but are NOT enforced.

🚀 Final Mental Model

Think of RLS as:

> Automatic invisible WHERE clause attached to your queries based on logged-in role.

If condition TRUE → row visible
If FALSE → row hidden

That's how specific rows are shown and others are completely hidden.

If you want, next I can also explain:

- How to debug RLS policies
- How to combine WITH CHECK for INSERT/UPDATE
- Or how this connects to Virtual Private Database (VPD) concept in theory

Just tell 👍

explain those codes we used for creating policies,
and how does the specific lines of codes shows only specific rows while hides others

Absolutely — let's dissect your RLS policies **line by line** and see exactly **how PostgreSQL filters rows** to show some and hide others. I'll use one policy at a time so it's crystal clear.

---

1. Employee Policy

SQL

```sql
CREATE POLICY employee_policy
ON employees
FOR SELECT
USING (
  current_setting('app.role', true) = 'Employee'
  AND emp_name = current_setting('app.username', true)
);
```

Line-by-line:

1. `CREATE POLICY employee_policy ON employees`
   - Defines a **row-level security policy** called `employee_policy` on the `employees` table.

Line-by-line:

1. `CREATE POLICY employee_policy ON employees`
   - Defines a **row-level security policy** called `employee_policy` on the `employees` table.
   - This policy applies to **rows in this table**, not the whole database.
2. `FOR SELECT`
   - This policy applies **only to SELECT queries**.
   - You could have different policies for INSERT, UPDATE, DELETE.
3. `USING (...)`
   - This is the **filter condition**: PostgreSQL will only **allow rows where the expression evaluates to TRUE**.
   - Rows where the expression is FALSE are **hidden**.
4. `current_setting('app.role', true) = 'Employee'`
   - Checks the **session variable** `app.role`.
   - Ensures this policy applies **only when the user is an employee**.
5. `AND emp_name = current_setting('app.username', true)`
   - Compares the row's `emp_name` to your session variable `app.username`.
   - Only the row **matching your username** will pass the filter; all others are automatically hidden.

Effect:

- Alice logs in as Employee → `app.username = 'Alice'` → Only the row with `emp_name = 'Alice'` is returned.
- Bob logs in → sees only his own row.

2. Manager Policy

SQL

```sql
CREATE POLICY manager_policy
ON employees
FOR SELECT
USING (
  current_setting('app.role', true) = 'Manager'
  AND department = current_setting('app.department', true)
);
```

Explanation:

1. Checks that the session role is `Manager`.
2. Filters rows where the `department` matches the manager's `app.department` session variable.

Effect:

- Manager of Sales → `app.department = 'Sales'` → sees all employees in Sales.
- Employees from other departments → hidden automatically.

---

3. HR Policy

SQL

Ask anything

Voice

give me output something like this

Perfect 👍 I understand — you want a **clean, formatted explanation output like in the screenshot**, with:

- Policy title
- SQL block
- Line-by-line explanation
- Clear "Effect" section

I'll structure it exactly like that for **all your RLS policies**.

1️⃣ Employee Policy

```sql
CREATE POLICY employee_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Employee'
    AND emp_name = current_setting('app.username', true)
);
```

Line-by-Line:

1. `CREATE POLICY employee_policy ON employees`

- Creates a Row Level Security policy.
- The policy name is `employee_policy`.
- It applies only to the `employees` table.

---

2. `FOR SELECT`

- This policy applies only to SELECT queries.
- It does not affect INSERT, UPDATE, DELETE.

---

3. `USING ( ... )`

- This is the **filter condition**.
- PostgreSQL will only return rows where this expression is TRUE.
- FALSE rows are automatically hidden.

---

4. `current_setting('app.role', true) = 'Employee'`

- Checks the session variable `app.role`.
- Policy works only if logged-in user's role is Employee.

---

5. `AND emp_name = current_setting('app.username', true)`

- Compares table column `emp_name`
- With session variable `app.username`.
- Only matching row passes.

🔎 Effect

If:

```
SET app.role = 'Employee';
SET app.username = 'Rahul';
```

Then internally PostgreSQL behaves like:

```
SELECT * FROM employees
WHERE emp_name = 'Rahul';
```

👉 Rahul sees only his own record.
👉 All other rows are hidden.

2️⃣ Manager Policy

```
CREATE POLICY manager_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Manager'
    AND department = current_setting('app.department', true)
);
```

Line-by-Line:

1. `CREATE POLICY manager_policy ON employees`

Creates policy for employees table.

---

2. `FOR SELECT`

Applies only to SELECT queries.

---

3. `current_setting('app.role', true) = 'Manager'`

Ensures this policy works only if user role is Manager.

---

4. `AND department = current_setting('app.department', true)`

Filters rows where:

- Employee's department
- Matches manager's department session variable

🔎 Effect

If:

```
SET app.role = 'Manager';
SET app.department = 'Sales';
```

PostgreSQL behaves like:

```
SELECT * FROM employees
WHERE department = 'Sales';
```

👉 Manager sees all employees in Sales.
👉 Other departments are hidden.

3️⃣ HR Policy

```
CREATE POLICY hr_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'HR'
    AND department = current_setting('app.department', true)
);
```

Line-by-Line:

- Applies only if role = HR
- Shows employees only in HR's department

🔎 Effect

If:

```
SET app.role = 'HR';
SET app.department = 'HR';
```

Internally becomes:

```
SELECT * FROM employees
WHERE department = 'HR';
```

👉 HR sees only HR department employees.

4️⃣ Admin Policy

```
CREATE POLICY admin_policy
ON employees
FOR SELECT
USING (
    current_setting('app.role', true) = 'Admin'
);
```

Line-by-Line:

- Checks if role is Admin.
- No department condition.
- No name condition.

🔎 Effect

If:

```
SET app.role = 'Admin';
```

Internally behaves like:

```
SELECT * FROM employees;
```

👉 Admin sees ALL rows.

🧠 How PostgreSQL Actually Filters Rows

When you run:

```
SELECT * FROM employees;
```

PostgreSQL internally rewrites it like:

```
SELECT * FROM employees
WHERE
   (Employee condition)
OR (Manager condition)
OR (HR condition)
OR (Admin condition);
```

But since only one role matches your session value,
only that policy condition becomes TRUE.

All other conditions become FALSE.

So only allowed rows pass.

🔐 Why Hidden Rows Truly Disappear

For every row:

- If USING condition = TRUE → row returned
- If FALSE → row completely removed from result

No error.
No warning.
It simply looks like those rows don't exist.

That’s the power of RLS.

If you want, I can now format this exactly in **lab-record write-up format** for your submission (since you prefer specific formatting style).